1. Resumen ejecutivo

En este documento se detalla el proceso de análisis y hallazgos de un binario de Unity con el global-metadata.dat protegido. Este tiene los exports del runtime renombrados, metadata embebido, cifrado y con la cabecera ofuscada (magic bytes y número de versión). Una vez completado el proceso se ha podido extraer el archivo global-metadata.dat para ser procesado junto con el GameAssembly.dll en Il2CppInspectorRedux. Se ha utilizado Ghidra para el análisis estático, ver como se leía global-metadata.dat y donde estaba y Frida para volcar el contenido de memoria con los datos del archivo en claro.

2. Alcance y autorización

Este análisis se ha llevado a cabo en una copia adquirida legalmente y en una máquina personal persiguiendo fines educativos y de investigación sobre técnicas de protección de binarios de Unity. No se distribuirán binarios, metadata o herramientas que permitan reproducir el proceso aquí descrito, tampoco se nombrará la aplicación analizada.

3. Contexto: primeras aproximaciones

El interés inicial por este binario surgió de una motivación personal, entender cómo se representaba el estado de la partida, que sirvió como punto de entrada al análisis. Ese objetivo quedó pronto en segundo plano: el trabajo derivó en el estudio de la protección de la metadata, que es lo que aquí se documenta.

La primera toma de contacto se intentó hacer a nivel de red interceptando el tráfico para después de aquí sacar la información deseada. Desafortunadamente el binario cuenta con certificate pinning lo que imposibilita este tipo de flujo de trabajo, ni siquiera forzando la API del sistema con Frida ya que para las comprobaciones utiliza funciones propias de Unity y están ofuscadas.

Tampoco se pudieron utilizar herramientas como Il2cppInspector para volcar la información de clases y funciones ya que global-metadata.dat se encuentra embebido y cifrado en el propio ejecutable.

A raíz de esto se decidió extraer el global-metadata.dat para luego volcar toda esta información sobre clases y funciones.

Vía intentada	Diagnóstico de entonces	Explicación tras este análisis
Interceptar el tráfico de red	Certificate pinning con validación propia que no cede forzando la API del sistema	TLS propio del motor (UnityTLS), que no pasa por la API de Windows
Análisis estático directo (IDA, Ghidra)	Stripping y ofuscación agresiva	Sin el metadata, el código nativo no tiene nombres
Navegar clases en runtime (frida-il2cpp-bridge)	No inicializa; probablemente símbolos no expuestos	Probable: busca los exports il2cpp_* por nombre, y están renombrados
Herramienta estándar (Il2CppInspector) y escaneo de memoria	Fallan sin resultado	Metadata cifrado y embebido, cabecera y magic alterados

4. Entorno y herramientas

Ítem	Valor
SO / arquitectura	Windows x64
Módulos analizados	GameAssembly.dll, UnityPlayer.dll
Versión de Unity	2022.3.62f2
Versión del metadata	31
Ghidra	11.4
Frida	17.2.11
Il2CppInspectorRedux	2026.2
Referencia de código	libil2cpp incluido en Unity Editor 2022.3.62

5. El objetivo: IL2CPP y la metadata

Unity permite compilar los juegos con dos backends de scripting distintos. Con Mono, el código C# se compila a bytecode de .NET, que conserva los nombres y es fácilmente recuperable con herramientas como dnSpy. Con IL2CPP, el caso de este binario, Unity convierte ese código C# primero a C++ y después lo compila a código máquina. El resultado es que la lógica del juego queda como un binario más, sin la información de tipos, clases y métodos que sí tendría un ensamblado .NET.

Esa información no desaparece, se almacena por separado en un archivo llamado global-metadata.dat. Ese archivo es el "mapa" que relaciona el código con los nombres originales de C#, es decir, qué dirección corresponde a cada método, cómo se llama cada clase y cada campo, qué cadenas usa el programa, etc. Herramientas como Il2CppInspector cruzan ese archivo con el binario GameAssembly.dll para reconstruir la estructura del proyecto, el árbol de clases y métodos con sus nombres.

Conviene distinguir dos cosas que la metadata permite recuperar de forma distinta. La estructura, nombres de clases, métodos y campos, y sus direcciones, se reconstruye directamente a partir de la metadata. La implementación, la lógica de cada método, sigue siendo código nativo, y recuperarla es un trabajo de ingeniería inversa aparte. Dicho de otro modo: la metadata te devuelve los nombres y el mapa, pero no el código fuente.

El archivo tiene una cabecera con una estructura conocida (Il2CppGlobalMetadataHeader). Empieza por un número mágico fijo, 0xFAB11BAF, seguido de un número de versión que indica el formato del archivo, y a continuación una serie de pares (offset, tamaño) que indexan las distintas secciones del archivo: la tabla de imágenes, la de tipos, la de métodos, las cadenas, etc. En este contexto, una imagen es cada uno de los ensamblados de C# que formaban el proyecto original (por ejemplo Assembly-CSharp.dll con el código del juego, mscorlib.dll, los módulos de UnityEngine, etc.); en este binario hay 169.

Precisamente por ser el elemento que hace legible el binario, la metadata es un objetivo habitual de las medidas anti-ingeniería-inversa. Un desarrollador que quiera dificultar el análisis puede cifrar el archivo, embeberlo dentro del propio ejecutable en lugar de dejarlo como fichero suelto, o alterar su cabecera. Sin la metadata en claro, las herramientas estándar no pueden reconstruir la estructura, y el analista se queda frente a un binario nativo sin nombres. Este fue el caso del objetivo analizado, y es lo que motivó el resto del trabajo.

Un detalle de esa cabecera resulta clave para entender cómo se pudo manipular. En las builds de release, la comprobación del número mágico y de la versión se realiza mediante IL2CPP_ASSERT, una macro que no genera ningún código en release. Es decir, el runtime en producción no verifica realmente que esos dos campos tengan los valores correctos, lo que permite a una protección falsearlos sin que el juego deje de funcionar.

6. Análisis estático

Dado que el binario estaba ofuscado la vía de buscar directamente il2cpp_init no era posible. La segunda vía más común, buscar por los magic bytes de global-metadata.dat, tampoco dio resultado. Más adelante se ve que parte de la cabecera del archivo global-metadata.dat, que es la que contiene los magic bytes y la versión, ha sido sobreescrita con basura.

Dado que los nombres de las funciones estaban ofuscados primero era necesario saber el nuevo nombre de il2cpp_init, para ello se ha utilizado UnityPlayer.dll usando la cadena “IL2CPP Root Domain” como ancla para encontrar el mapeo entre il2cpp_init y su nombre después de haber sido ofuscada.

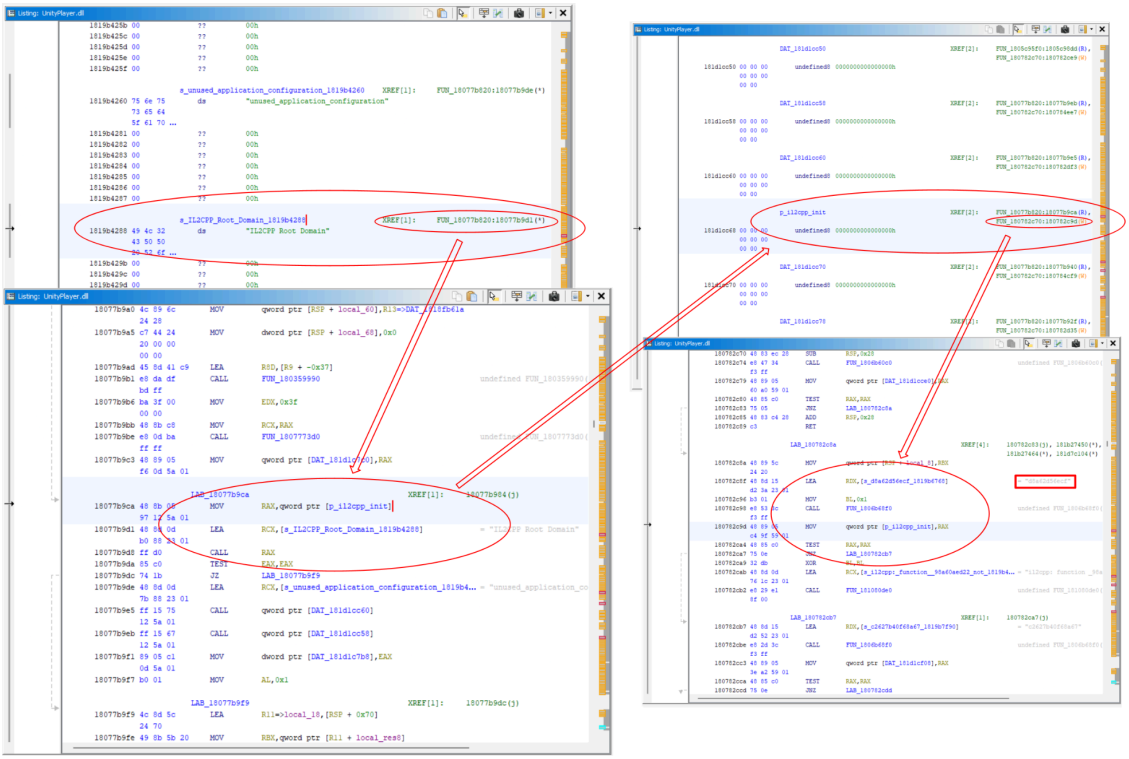


Figura 1: la cadena IL2CPP Root Domain en UnityPlayer.dll lleva al puntero donde se guarda el export renombrado de il2cpp_init.

Una vez se ha conseguido esta información se han trazado las llamadas que Unity hace a la hora de cargar global-metadata.dat:

il2cpp_init
-> Runtime::Init
-> MetadataCache::Initialize
-> GlobalMetadata::Initialize

-> MetadataLoader::LoadMetadataFile

Hasta llegar a la función MetadataLoader::LoadMetadataFile. Si se analiza toda la traza se puede ver cómo los desarrolladores han sobrescrito LoadMetadataFile con su rutina propia. La función en sí sigue retornando lo mismo así que es posible obtener el puntero al inicio de la información si se sabe la dirección de memoria de esta.

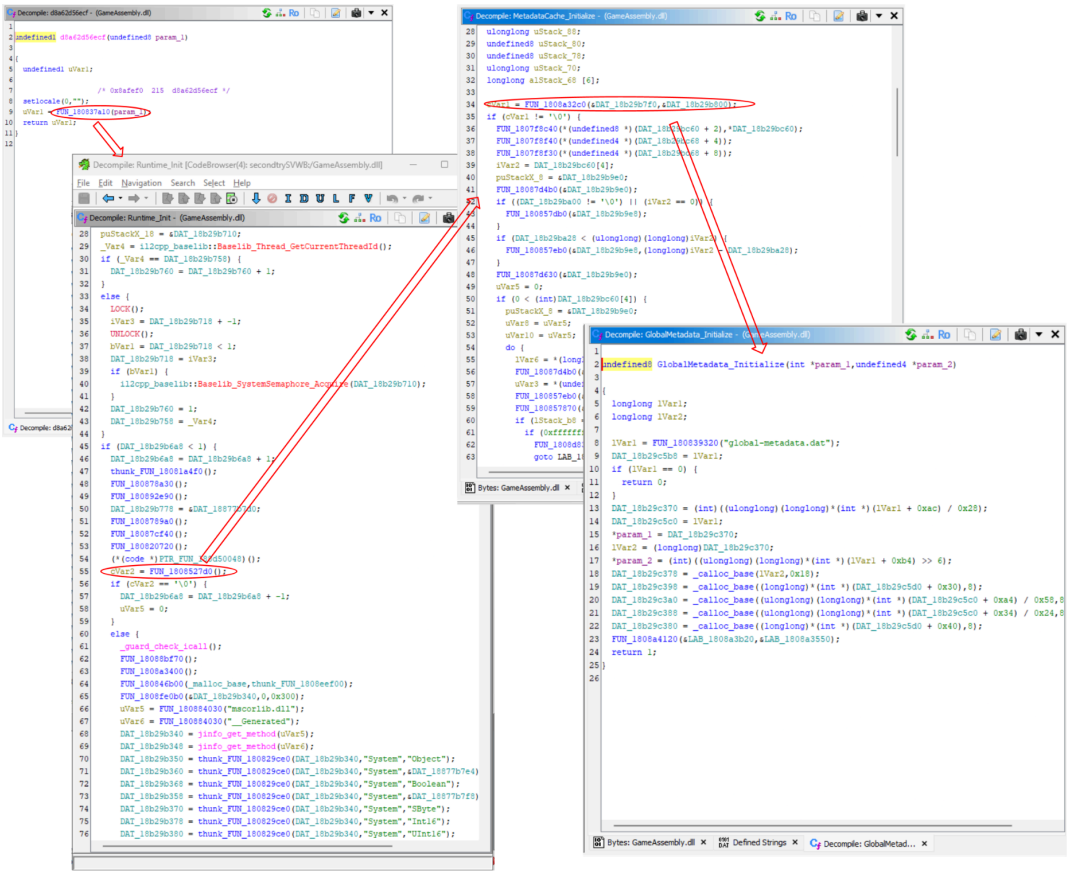


Figura 2: traza en GameAssembly.dll desde il2cpp_init hasta GlobalMetadata::Initialize, que llama a la función de carga sustituida.

7. Anatomía de la protección

Medida	Evidencia	Por qué no rompe el juego	Cómo se evitó
Exports del runtime renombrados	Hay muchos con nombres “aleatorios”	UnityPlayer pide los nombres nuevos	Seguir el uso desde UnityPlayer
Nombres coordinados en UnityPlayer	GetProcAddress con el nombre ofuscado	Windows busca el nombre exacto	Leer el nombre que se resuelve
Mensajes de error señuelo	El error nombra otra función	Solo se imprime si falla	Fiarse del flujo de datos, no de cadenas
Loader sustituido por trampolín	Argumentos a cero y salto al código propio	Mantiene la interfaz	Leer el retorno
Metadata cifrado y embebido	Blob en el DLL, transformación SIMD, caché global	Se descifra antes de usarse	Leer el puntero ya descifrado
Función anti-análisis	~34 KB, descompilador atascado, constantes en XOR	No afecta a la ejecución	Tratarla como caja negra
Magic y versión falseados	a239cb05 y valor sin sentido	Solo los comprueba IL2CPP_ASSERT	Restaurarlos a partir del código fuente

8. Extracción dinámica

Una vez encontrada la función MetadataLoader::LoadMetadataFile, la extracción de manera estática queda descartada ya que global-metada.dat está cifrado y la función de descifrado es tan grande que no acaba de cargar además de ser compleja. La vía más rápida es extraer de manera dinámica la información una vez ha sido descifrado por el propio programa. Teniendo las direcciones de memoria de las funciones y variables de interés, hacer esto con Frida es directo.

En este caso particular tenemos 2 vías por las que volcar la información: el valor que retorna la propia función MetadataLoader::LoadMetadataFile o bien la variable donde se guarda el puntero a los datos descifrados después de la primera lectura. Inicialmente se iban a explorar ambas opciones pero el hecho de tener una variable con la información disponible en cualquier momento facilita mucho las cosas ya que no hay que engancharse al programa desde el inicio. En este caso particular, como ASLR está activado en el sistema, si queremos saber la dirección absoluta del puntero, necesitamos sumar la base del ejecutable calculada de manera dinámica en el propio script de Frida con la dirección relativa que se puede calcular en Ghidra restando la base de la imagen a la dirección de memoria deseada.

Antes de extraer la información, en el propio script de Frida, se ha analizado la cabecera tanto para saber el tamaño de los datos a volcar como para ver si los datos estaban descifrados en este punto. La cabecera se valida por su estructura, ya que el magic no es fiable. Aquí también comprobamos por qué la búsqueda por el magic bytes no funcionó, estos han sido sustituidos por basura.

Comprobación	**Resultado**
Magic bytes	0xa239cb05
Versión	0xd9c131d8 (-641650216)
Secciones de la cabecera	31 pares contiguos, sin huecos, desde 0x100
Imágenes (0x1a68 / 0x28)	169, división exacta

Para extraer el tamaño de los datos a volcar se ha obtenido el valor de offset+tamaño más grande que encontramos en la cabecera siendo este 0x1E70938 (~30,4MB).

Una vez se ha volcado la información, para poder utilizarla en Il2CppInspector hace falta restaurar los magic bytes y el número de versión. Lo más sencillo es hacer ambas cosas a la vez basándose en el código fuente de Unity. Para ello se comprueba la versión del ejecutable, 2022.3.62f2, y en base a eso se comprueban los datos de interés en el código fuente de Unity.



Figura 3: versión de Unity en las propiedades de UnityPlayer.dll y GlobalMetadata::Initialize en libil2cpp, con las comprobaciones IL2CPP_ASSERT de la cabecera.

9. Recomendaciones defensivas

La protección analizada combina varias capas, y cada una cumple su función frente a un tipo concreto de análisis. Sin embargo, el proceso descrito muestra que bastó con encontrar un punto débil en la cadena para obtener el resultado final sin romper ninguna de las demás. Las siguientes recomendaciones se centran en esos puntos débiles, ordenadas según el peso que tuvieron en el análisis.

Proteger también las cadenas del motor. Renombrar los exports del runtime impide localizar il2cpp_init por su nombre, pero la medida pierde gran parte de su eficacia cuando las cadenas que rodean esas funciones siguen en claro. Cadenas como “IL2CPP Root Domain” en UnityPlayer.dll o “global-metadata.dat” sirvieron de ancla para llegar desde UnityPlayer.dll al nombre ofuscado y, a partir de ahí, a toda la cadena de carga del metadata. Cifrar estas cadenas, o eliminarlas cuando no son necesarias en tiempo de ejecución, obliga a reconstruir ese camino por otros medios mucho más costosos.

No dejar el metadata descifrado de forma permanente en una variable global. Este fue el punto decisivo. El buffer descifrado queda referenciado desde s_GlobalMetadata durante toda la ejecución, de modo que no es necesario engancharse al proceso desde el inicio ni interceptar el descifrado: basta con leer un puntero en cualquier momento. Hay dos alternativas que elevan el coste del volcado:

- Descifrar el metadata por secciones y solo cuando se necesitan, de forma que nunca exista una copia completa en claro en memoria;
- Mover los valores de la cabecera a una estructura propia y borrar o volver a ofuscar la cabecera del buffer tras la inicialización, para que el volcado no sea directamente utilizable.

Ambas opciones exigen modificar el runtime más a fondo que la sustitución de LoadMetadataFile, y tienen un impacto en rendimiento que habría que medir.

Alterar la estructura, no solo los identificadores. Falsear el magic y la versión basta para frustrar la búsqueda por firma y la carga directa en herramientas estándar, pero la estructura de la cabecera queda intacta. Los pares (offset, tamaño) contiguos desde 0x100 y las divisiones exactas por el tamaño de cada estructura permitieron validar el volcado sin depender del magic. Además, la versión correcta se pudo recuperar del propio código fuente de libil2cpp. Reordenar los campos de la cabecera o permutar el orden de las secciones, adaptando el runtime en consecuencia, obligaría al analista a reconstruir el formato antes de poder usar cualquier herramienta.

Proteger el resultado, no solo la rutina. La función de descifrado, de unos 34 KB, con constantes enmascaradas mediante XOR y capaz de atascar el descompilador, es probablemente la capa más elaborada del esquema. Sin embargo, no hizo falta entenderla: el análisis la trató como una caja negra y se limitó a leer su salida. El esfuerzo invertido en dificultar el análisis estático de una rutina pierde valor si su resultado queda expuesto en memoria. Por eso esta capa debería ir acompañada de las medidas anteriores.

Detectar la instrumentación en tiempo de ejecución. Todo el volcado se apoyó en la capacidad de leer la memoria del proceso con Frida. Comprobar la presencia de instrumentación dinámica (módulos inyectados, hilos ajenos, modificaciones en funciones críticas) no impide el análisis, pero obliga a añadir una fase de evasión antes de cualquier extracción.

Los señuelos solo frenan a quien sigue las cadenas. Los mensajes de error que nombran funciones falsas son útiles contra un análisis superficial basado en búsquedas de texto. Frente a un análisis que sigue el flujo de datos (quién escribe un puntero, quién lo lee y qué se hace con el valor de retorno) no aportan ninguna resistencia, y no deberían considerarse una medida de protección por sí mismos.

Asumir que la protección del lado del cliente es temporal. El código y los datos que el cliente necesita para funcionar acaban, en algún momento, en claro en la memoria de una máquina controlada por el usuario. Ninguna combinación de las medidas anteriores impide la extracción; lo que hacen es aumentar el tiempo y el conocimiento necesarios para conseguirla. Por eso la información cuya integridad sea realmente crítica, como el estado de la partida o las validaciones relevantes, debería residir o verificarse en el servidor, y no depender de que el cliente permanezca opaco.

10. Trabajo futuro: qué permite la estructura recuperada

Con el metadata restaurado y procesado, el binario deja de ser código nativo sin nombres y pasa a tener un mapa completo de clases, métodos, campos y cadenas. Esto no es un fin en sí mismo, sino el punto de partida para análisis que antes no eran viables. Este apartado recoge las líneas de trabajo que se abren a partir de aquí, y respetan el mismo alcance definido en el apartado 2: análisis local, con fines educativos y sin interactuar con los servicios en línea del juego.

Aplicar los símbolos al análisis estático. Il2CppInspectorRedux puede generar scripts para Ghidra que renombran las funciones de GameAssembly.dll, aplican los tipos de C# reconstruidos y etiquetan las cadenas literales. Tras aplicarlos, el siguiente paso natural es validar el resultado comprobando métodos al azar: que el nombre asignado sea coherente con lo que hace la función y que los accesos a campos coincidan con los offsets de la estructura recuperada. Con eso se confirmaría que el volcado es correcto más allá de la validación estructural del apartado 8.

Retomar el objetivo inicial. El análisis empezó con el interés de entender cómo se representaba el estado de la partida, y ese objetivo quedó aparcado por la protección. Con la estructura disponible, se pueden identificar las clases que modelan ese estado y estudiar cómo se organizan sus campos y qué métodos los modifican. Esta información puede obtenerse de forma pasiva, leyendo la memoria del propio proceso con Frida, sin alterar la ejecución ni la comunicación con el servidor.

Instrumentación en tiempo de ejecución a nivel de C#. frida-il2cpp-bridge no inicializaba porque busca los exports il2cpp_* por su nombre original. Ahora que se conoce la correspondencia entre esos nombres y los nombres ofuscados, debería ser posible proporcionársela a la herramienta, o resolver esas funciones manualmente, para trabajar directamente con clases y métodos en lugar de direcciones. Esto convertiría la instrumentación dinámica de un trabajo con offsets a uno con nombres, mucho más legible y robusto ante actualizaciones del binario.

Estudiar la rutina de descifrado con el resultado conocido. La función de descifrado se trató como una caja negra porque no hacía falta entenderla. Sin embargo, ahora se dispone tanto del blob cifrado embebido en el DLL como de su resultado en claro, lo que facilita mucho su análisis: se puede verificar cada etapa intermedia contra datos conocidos. Entender el algoritmo permitiría determinar si la clave es fija o se deriva de algún valor del binario, y por tanto si el mismo esquema se repite entre versiones.

Comprobar la reproducibilidad entre versiones. Repetir el proceso sobre una actualización posterior del juego permitiría medir cuánto del trabajo es reutilizable. Por ejemplo: si los nombres ofuscados de los exports cambian en cada compilación, si la variable global sigue en el mismo lugar relativo y si el falseamiento de la cabecera usa los mismos valores. Si el esquema resulta estable, el proceso podría automatizarse en un único script que localice el puntero, vuelque el metadata y restaure la cabecera.

Entender la validación de certificados. La interceptación del tráfico fue la primera vía bloqueada. Con la estructura disponible, se pueden localizar las clases de Unity responsables de la validación TLS y documentar cómo implementa el motor el certificate pinning por su cuenta, sin pasar por la API de Windows. El objetivo sería explicar por qué fallaron los intentos iniciales, no interceptar ni modificar la comunicación con el servidor.

Identificar la protección. Los rasgos observados forman una firma reconocible: exports renombrados de forma coordinada entre UnityPlayer.dll y GameAssembly.dll, cargador sustituido por un trampolín, transformación SIMD y cabecera falseada con IL2CPP_ASSERT como única comprobación. Compararla con soluciones comerciales de protección para Unity podría determinar si se trata de un producto conocido o de una implementación propia, y en el primer caso, qué otras medidas suele incluir.

Todas estas líneas comparten el mismo límite: el análisis se realiza sobre una copia local, sin modificar el comportamiento del juego en partidas en línea, sin interactuar con los servidores y sin distribuir nada de lo extraído.